Assignment 1 - Patient Intake and Queue Tracking

Healthcare Domain: CareFlow

Submission overview. This document presents the scope and requirements, domain model, design model, SOLID decision and ADR, AI critique, and final reflection/team handoff for the patient-facing slice of CareFlow.

Part	Section
1	Scope and Requirements Slice
2	Domain Model
3	Design Model
4	SOLID Decision and ADR
5	AI Setup and Critique
6	Reflection and Team Handoff

Part 1: Scope and Requirements Slice

Slice name

Patient Intake and Queue Tracking

In scope

A patient can complete and edit an intake form, submit a completed form for nurse review, and, after being placed in the triage queue, view their current queue position and estimated wait time.

Out of scope

User authentication and patient account management; nurse review of submitted intake forms; assignment or modification of triage priority; queue reordering logic; clinician evaluation; and updating a patient's status after clinical evaluation.

Requirements

- **R1.** As a patient, I can complete and submit an intake form so that my information is available for nurse review before I enter the triage queue.
- **R2.** As a patient, I can review and edit my intake form before submission so that I can correct or update my information.
- **R3.** As a patient in the triage queue, I can view my current queue position so that I know where I am in the waiting process.
- **R4.** As a patient in the triage queue, I can view an estimated wait time so that I have an indication of how long I may wait before being seen.

Rules / Invariants

- **Intake completeness (R1):** An intake form cannot be submitted unless all required patient and visit information is complete and valid.
- **Submission lock (R1, R2):** Once an intake form has been submitted for nurse review, the patient cannot modify it.
- **Queue eligibility (R1, R3):** Submitting an intake form does not automatically place the patient in the triage queue. The patient must first be reviewed and assigned a priority by a nurse.
- **Queue visibility (R3, R4):** A patient can view queue position and estimated wait information only after being placed in the triage queue.
- **Dynamic queue position (R3, R4):** The displayed position must reflect the patient's current position in the triage queue and may change as the queue is updated.
- **Wait estimate (R4):** Estimated wait time is based on the patient's current queue position and the average time required to see a patient.
- **Patient data access (R1-R4):** A patient can access only their own intake form and queue information.

Part 2: Domain Model

Entities and Key Attributes

Entity	Key conceptual attributes
Patient	patient ID; name; date of birth; contact information
IntakeForm	intake ID; gender; insurance name; insurance number; reason for visit; submission status; submitted at
QueueEntry	entry ID; priority; entered at
TriageQueue	queue ID; average service time

Relationships with Multiplicity

- **Patient "1" - "0..1" IntakeForm: completes.** A patient may have zero or one current intake form; each intake form belongs to exactly one patient.
- **Patient "1" - "0..1" QueueEntry: has.** A patient may have no queue entry while completing intake or awaiting nurse review; once placed in the queue, the patient has one current entry.
- **TriageQueue "1" - "0..*" QueueEntry: contains.** A triage queue may contain zero or many entries; each entry belongs to one queue.

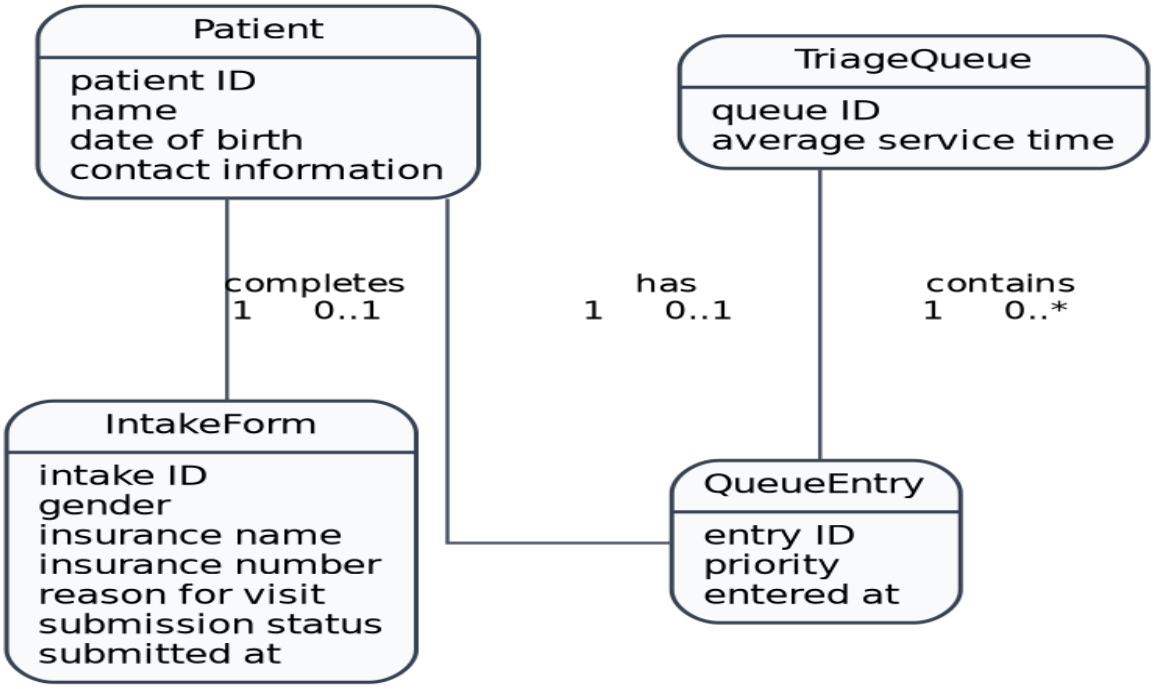


Figure 1. Domain class diagram for the patient-facing slice.

Design Note

QueueEntry represents a patient's membership in the current triage queue. Priority belongs to the queue entry rather than the patient because priority applies to the patient's current visit and may differ between visits. Current queue position is not stored as an attribute; it is derived from the ordering of entries within TriageQueue, allowing the displayed position to remain current as the queue changes. Estimated wait time is likewise derived from the current position and the queue's average service time rather than stored as a fixed value.

Part 3: Design Model

Design Class Diagram

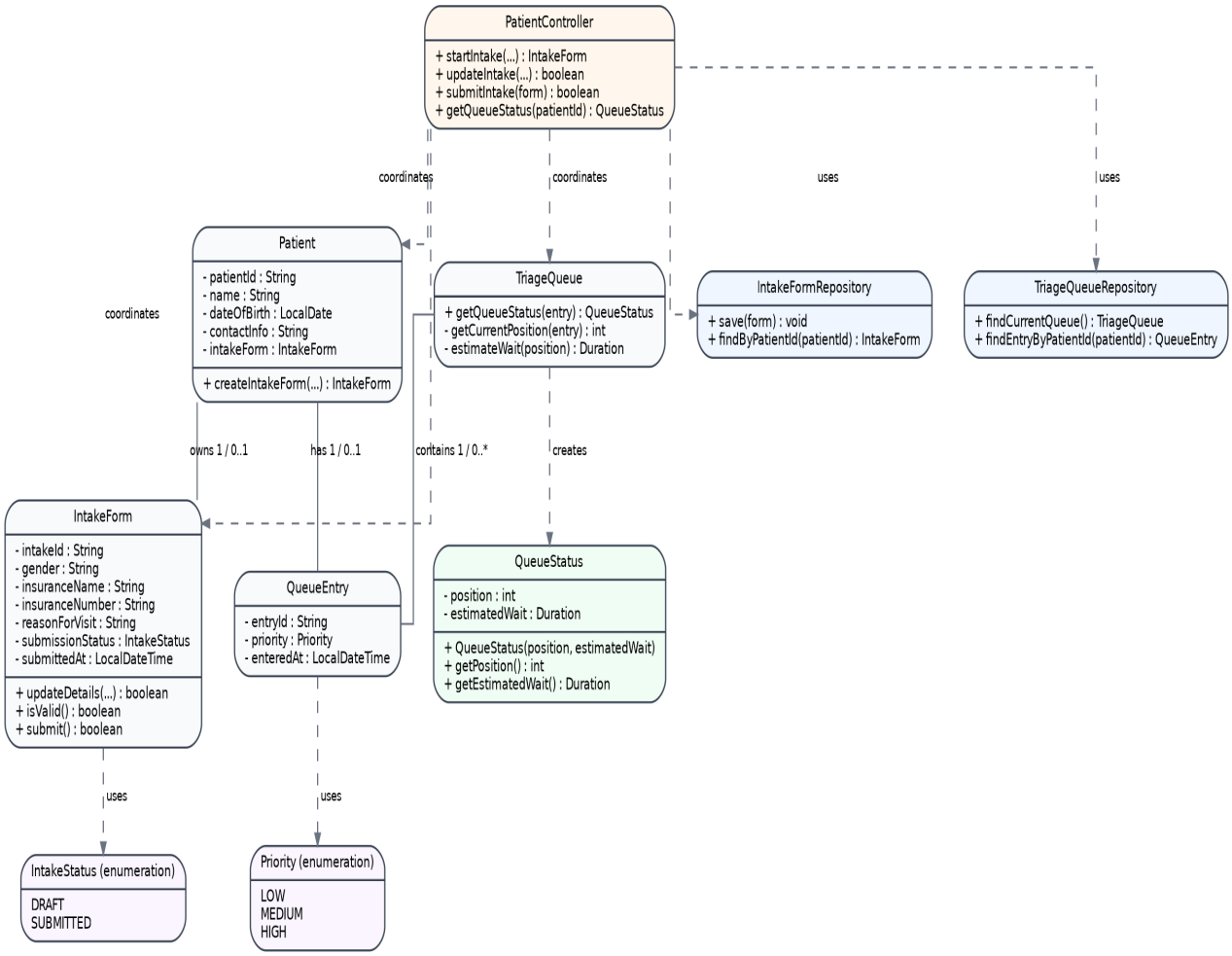


Figure 2. Design-level class diagram with Java types, operations, controller, repositories, value object, and enums.

Responsibility Table

Behavior	Owner class	GRASP reasoning
Create an intake form	Patient	Creator: Who contains or closely owns the created IntakeForm?
Update intake details	IntakeForm	Information Expert: Who owns the data and knows whether editing is allowed?
Validate required intake information	IntakeForm	Information Expert: Who has the information needed to determine completeness?
Submit and lock an intake form	IntakeForm	Information Expert: Who knows validity and current submission state?
Coordinate patient intake requests	PatientController	Controller: Who receives the patient-facing system event and delegates domain work?
Determine current queue position	TriageQueue	Information Expert: Who knows the ordered collection of queue entries?
Calculate estimated wait	TriageQueue	Information Expert: Who knows current position and average service time?

Behavior	Owner class	GRASP reasoning
Create queue status	TriageQueue	Creator: Who has the information needed to initialize QueueStatus?
Coordinate queue-status requests	PatientController	Controller: Who receives the request and coordinates retrieval/calculation?
Persist/retrieve intake forms	IntakeFormRepository	Data responsibility: isolates intake-form persistence from the domain.
Retrieve current queue information	TriageQueueRepository	Data responsibility: retrieves the current queue and patient queue entry.

Layer Map

Layer	Classes
Presentation	PatientController
Domain	Patient; IntakeForm; QueueEntry; TriageQueue; QueueStatus; IntakeStatus; Priority
Data	IntakeFormRepository; TriageQueueRepository

Key Design-Level Signatures

Patient

```
public IntakeForm createIntakeForm(
    String gender,
    String insuranceName,
    String insuranceNumber,
    String reasonForVisit
);
```

IntakeForm

```
public boolean updateDetails(
    String gender,
    String insuranceName,
    String insuranceNumber,
    String reasonForVisit
);
public boolean isValid();
public boolean submit();
```

TriageQueue and QueueStatus

```
public QueueStatus getQueueStatus(QueueEntry entry);
private int getCurrentPosition(QueueEntry entry);
private Duration estimateWait(int position);

public QueueStatus(int position, Duration estimatedWait);
public int getPosition();
public Duration getEstimatedWait();
```

PatientController

```
public IntakeForm startIntake(Patient patient, String gender,
    String insuranceName, String insuranceNumber, String reasonForVisit);
public boolean updateIntake(IntakeForm form, String gender,
    String insuranceName, String insuranceNumber, String reasonForVisit);
public boolean submitIntake(IntakeForm form);
public QueueStatus getQueueStatus(String patientId);
```

Data Layer

```
public void IntakeFormRepository.save(IntakeForm form);
public IntakeForm IntakeFormRepository.findByPatientId(String patientId);
public TriageQueue TriageQueueRepository.findCurrentQueue();
public QueueEntry TriageQueueRepository.findEntryByPatientId(String patientId);
```

Queue-Status Interaction

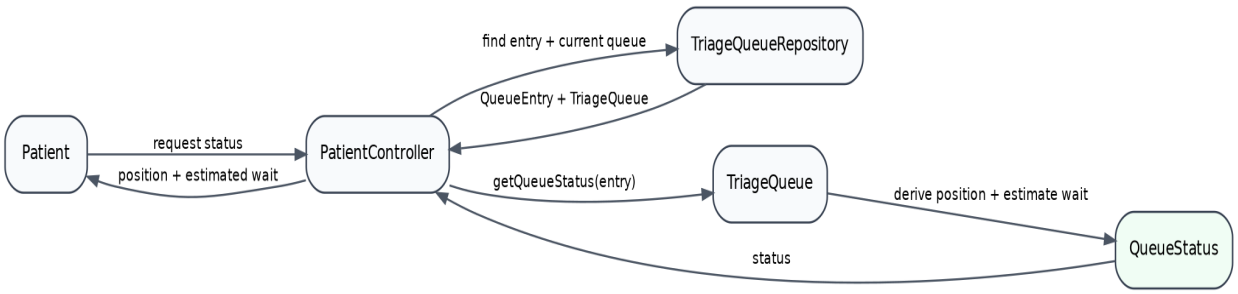


Figure 3. Simplified sequence of the queue-status request.

Part 4: SOLID Decision and ADR

Principle in focus: Single Responsibility Principle (SRP)

Decision path: Applied by design

Likely Change

The logic used to determine a patient's queue position and estimated wait may change as CareFlow's queue rules evolve. The current estimate uses queue position and average service time, but future requirements may change how the estimate is calculated. Queue-status logic should be able to change without affecting patient information or presentation-layer coordination.

GRASP Reasoning

The primary GRASP principle considered was **Information Expert**: Which class has the information necessary to determine a patient's current queue position and estimated wait? TriageQueue owns the ordered collection of QueueEntry objects and the average service time. Therefore, it has the information necessary to determine the current position and estimated wait. This also supports SRP by keeping queue-related calculations within TriageQueue while PatientController remains responsible for coordination.

Relevant Code

```
public class TriageQueue {
    private String queueId;
    private Duration averageServiceTime;
    private List<QueueEntry> entries;

    public QueueStatus getQueueStatus(QueueEntry entry) {
        int position = getCurrentPosition(entry);
        Duration estimatedWait = estimateWait(position);
        return new QueueStatus(position, estimatedWait);
    }

    private int getCurrentPosition(QueueEntry entry) {
        return entries.indexOf(entry) + 1;
    }

    private Duration estimateWait(int position) {
        return averageServiceTime.multipliedBy(position - 1);
    }
}

public final class QueueStatus {
    private final int position;
    private final Duration estimatedWait;

    public QueueStatus(int position, Duration estimatedWait) {
        this.position = position;
        this.estimatedWait = estimatedWait;
    }

    public int getPosition() { return position; }
    public Duration getEstimatedWait() { return estimatedWait; }
}
```

ADR-001: Keep Queue-Status Logic in TriageQueue

Status: Accepted **Date:** 2026-10-02

Context

CareFlow must provide patients with their current queue position and estimated wait time. The current estimate uses the patient's position in the ordered queue and the queue's average service time. This calculation may change as queue and wait-time rules evolve. Placing this logic in Patient would give the patient responsibility for information it does not own, while placing it in PatientController would mix domain calculations with presentation coordination.

Decision

Apply the **Single Responsibility Principle** by keeping queue-status calculations in TriageQueue. TriageQueue determines the patient's current position from its ordered QueueEntry collection, calculates the estimated wait, and returns the result as a QueueStatus. This decision also follows **GRASP Information Expert**, because TriageQueue owns the information required to perform these calculations. PatientController remains responsible for coordinating the request and delegating the domain work.

Consequences

- Queue-position and wait-time rules remain separate from patient and presentation responsibilities.
- Queue calculations stay with the object that owns the information needed to perform them.
- PatientController remains focused on coordinating patient requests.
- Changes to the current queue-status calculation are localized to the queue domain.
- TriageQueue gains responsibility for multiple queue-related calculations and must remain cohesive as the domain evolves.
- Changes to the queue-status calculation still require modifying TriageQueue.

Part 5: AI Setup and Critique

Prompt Used

I am designing a healthcare patient intake and triage queue system. Review the following Java design and recommend changes that would make it more SOLID and extensible. Identify which SOLID principles your recommendations address and provide revised code where appropriate. Focus on the design as given; do not assume additional requirements unless you clearly identify them as assumptions.

Relevant AI Output (excerpt)

The AI recommended extracting the wait-time calculation into a strategy and supplied the following representative code:

```
public interface WaitTimeEstimator {
    Duration estimateWait(List<QueueEntry> entriesAhead);
}

public final class AverageServiceTimeEstimator
        implements WaitTimeEstimator {
    private final Duration averageServiceTime;

    public AverageServiceTimeEstimator(Duration averageServiceTime) {
        Objects.requireNonNull(averageServiceTime, "averageServiceTime");
        if (averageServiceTime.isNegative()) {
            throw new IllegalArgumentException(
                "averageServiceTime must not be negative");
        }
        this.averageServiceTime = averageServiceTime;
    }

    @Override
    public Duration estimateWait(List<QueueEntry> entriesAhead) {
        return averageServiceTime.multipliedBy(entriesAhead.size());
    }
}
```

The AI also proposed QueueStatusProvider and, based on possible future requirements, QueueOrderingPolicy and QueueManager abstractions. It identified SRP, OCP, DIP, ISP, and LSP as relevant principles.

AI Critique

The AI correctly identified several potential design improvements, but some recommendations introduce abstractions beyond the requirements of my current slice. The WaitTimeEstimator strategy applies SRP, OCP, and DIP by separating wait-time calculation from TriageQueue. However, CareFlow currently has only one defined calculation based on queue position and average service time. I would therefore defer this abstraction until a second concrete estimation method creates a demonstrated need for variation. I would also reject the proposed QueueStatusProvider, QueueOrderingPolicy, and QueueManager interfaces for this slice. These introduce additional indirection, and some depend on future requirements that the AI itself identified as assumptions rather than current requirements.

I would accept the AI's correctness improvements. The current `indexOf()` logic does not handle a QueueEntry that is absent from the queue, potentially producing position zero and a negative wait time. I would explicitly handle this case and validate that QueueStatus cannot contain a position below one or a negative estimated wait. These changes strengthen the current design without introducing abstractions that are not yet justified by CareFlow's requirements.

Part 6: Reflection and Team Handoff

Reflection

The **Single Responsibility Principle (SRP)** mattered most in my design because the patient workflow crosses several responsibilities that could easily have been placed in the wrong classes. Using SRP together with GRASP helped establish clear ownership. Patient creates and owns its current IntakeForm, while IntakeForm is responsible for validating, updating, and submitting its own data. TriageQueue owns the queue information needed to derive a patient's current position and estimated wait, and PatientController coordinates these actions without taking responsibility for domain rules.

The most important design decision was keeping queue-status calculations within TriageQueue. Since it owns the ordered QueueEntry collection and average service time, this placement follows GRASP Information Expert while keeping queue logic out of Patient and the presentation layer.

The AI critique also reinforced that applying SOLID does not require creating an abstraction for every possible future change. Although a WaitTimeEstimator strategy could support multiple estimation algorithms, the current requirements define only one. Keeping that calculation in TriageQueue avoids unnecessary indirection while leaving room to refactor if a second concrete algorithm is introduced. This balance allowed the design to remain cohesive and extensible without over-engineering the current MVP.

Team Handoff

The strongest artifact to carry into **Checkpoint 1** is the design model and its responsibility assignments, particularly the separation among Patient, IntakeForm, QueueEntry, and TriageQueue. It provides a clear foundation for integrating the patient-facing intake and queue-status workflows with the nurse and clinician slices.